

SOFTWARE REQUIREMENTS SPECIFICATION

ExamEnv

Infrastructure-as-Code for Exams — One-Click, Reproducible Exam Environments

|                   |                                                                                |
|-------------------|--------------------------------------------------------------------------------|
| Document Version  | 1.0 (Draft)                                                                    |
| Prepared For      | Software Design Principles (SDP) Project Submission                            |
| Domain            | EdTech / DevOps / Cloud Infrastructure                                         |
| Core Technologies | FastAPI, PostgreSQL, Celery + Redis, Docker, WebSockets, LLM-based PDF Parsing |

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of ExamEnv, a platform that converts teacher-supplied exam environment requirements into a structured, machine-readable specification and automates the provisioning of isolated, reproducible student environments. This SRS covers the complete end-to-end system: the teacher-facing dashboard, the AI/LLM parsing pipeline, the student provisioning agent, and the real-time monitoring dashboard.

1.2 Document Conventions

Functional requirements are labeled REQ-<feature>-<number> (for example, REQ-UPLOAD-1) and are listed in priority order within each feature, with higher-priority requirements appearing first. Terms defined in Appendix A (Glossary) are used consistently throughout the document.

1.3 Intended Audience and Reading Suggestions

This document is intended for the development team (ML/Parser Engineer, Backend Developer, DevOps/Agent Developer), project evaluators/instructors, and, at a high level, prospective teacher and student users. Readers new to the project should start with Section 2 (Overall Description) for context, then proceed to Section 4 (System Features) for the core functional requirements, and refer to Section 5 for non-functional requirements such as security and performance.

1.4 Product Scope

ExamEnv eliminates the time lost to manual environment setup before labs and exams. Teachers upload exam requirements as a PDF; an AI parser extracts packages, versions, environment variables, ports, datasets, and setup commands into a YAML Infrastructure-as-Code (IaC) spec, which the teacher reviews and publishes. Students then run a lightweight CLI agent that previews the changes (dry-run), builds a Docker container from the spec, verifies that all dependencies are correctly installed, and reports status back to the platform. Teachers see live, per-student readiness status (Ready / Failed / Not Started) on a real-time dashboard and can inspect error logs to help students debug failures.

1.5 References

Project idea document: 'ExamEnv — Infrastructure-as-Code for Exams' (internal project proposal). SRS structure adapted from the IEEE-style SRS template (Karl E. Wiegers).

## 2. Overall Description

### 2.1 Product Perspective

ExamEnv is a new, self-contained, multi-component platform rather than a follow-on to an existing product. It consists of four cooperating zones: the Teacher Dashboard (Zone 1), the Backend Hub (Zone 2, built on FastAPI with a PostgreSQL database and a Celery/Redis task queue), the ML Parser (Zone 3, an LLM-based worker), and the Student Agent (Zone 4, a Python CLI that drives the local Docker engine). The diagram below illustrates the major components and their interconnections.

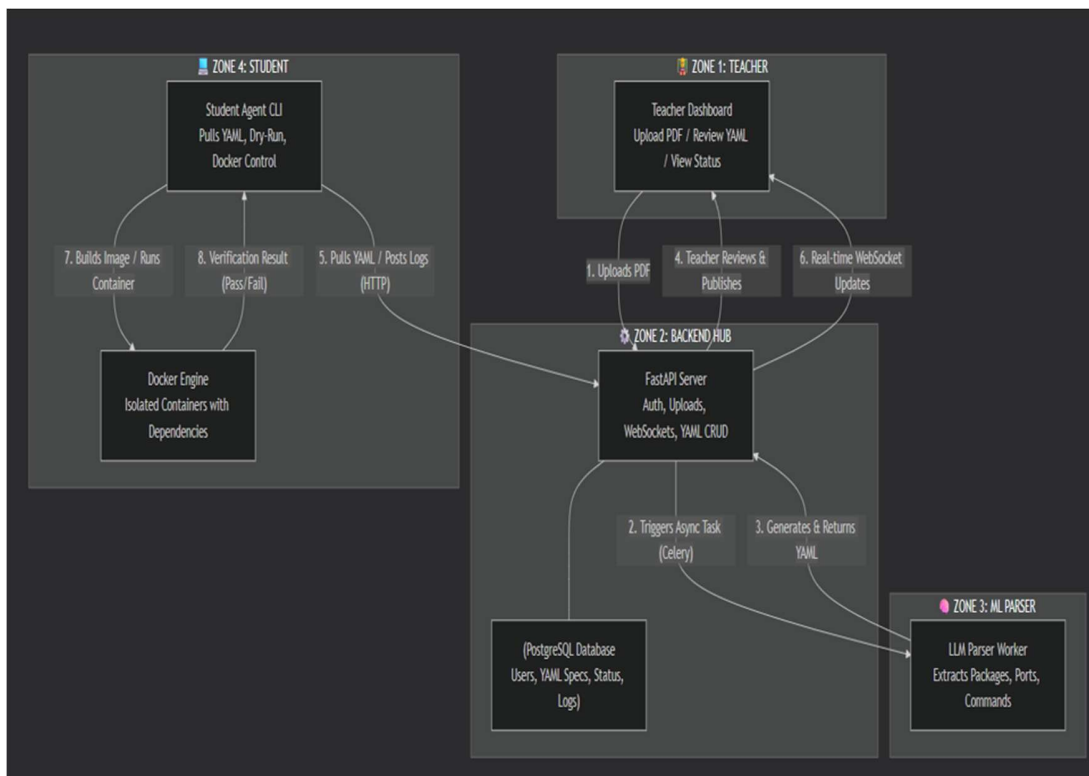


Figure 1: ExamEnv system architecture and data flow across the four zones.

### 2.2 Product Functions

- Upload exam requirement PDFs and trigger automated AI parsing into a YAML IaC spec.
- Review, manually correct, and publish the generated YAML specification.
- Pull a published environment spec via a student CLI agent and preview changes before applying them (dry-run).
- Build and run an isolated Docker container matching the published spec, with dataset/code volume mounts.
- Automatically verify installed dependencies inside the container and report pass/fail status.
- Display real-time, per-student readiness status to teachers via a WebSocket-driven dashboard.
- Allow teachers to inspect detailed error logs for failed environments and assist students in debugging.

### 2.3 User Classes and Characteristics

- **Teacher:** Primary content owner. Uploads requirement documents, reviews/edits AI-generated YAML, publishes environments, and monitors class-wide readiness. Assumed to have domain expertise but not necessarily deep DevOps knowledge.
- **Student:** End user of the provisioning agent. Runs CLI commands to fetch, preview, and provision their exam environment. Assumed to have basic command-line familiarity but not necessarily Docker expertise.
- **System Administrator / DevOps:** Manages backend deployment, the task queue, database, and the template library of pre-built YAML specs and verification scripts. Requires full technical access.

## 2.4 Operating Environment

The Teacher Dashboard runs as a web application accessible from any modern browser. The Backend Hub (FastAPI, PostgreSQL, Celery/Redis) is deployed on a cloud virtual machine. The Student Agent is a cross-platform Python CLI tool that must run on a machine with Docker Engine installed and sufficient permissions to build and run containers.

## 2.5 Design and Implementation Constraints

- Student machines must have Docker Engine installed and running; environments cannot be provisioned without it.
- Accuracy of the generated YAML spec depends on the quality and structure of the uploaded PDF; ambiguous requirement documents may require manual teacher correction.
- Asynchronous parsing requires a functioning Celery + Redis message queue; parsing delays are expected under high upload volume.
- Real-time dashboard updates depend on a persistent WebSocket connection between the backend and the teacher's browser.
- The system depends on an external LLM service for PDF parsing, introducing a third-party API dependency.

## 2.6 User Documentation

The platform will ship with a Student Agent CLI quick-start guide (installation, authentication, and the pull / dry-run / provision command sequence) and a Teacher Dashboard onboarding guide (uploading requirements, reviewing/editing YAML, publishing, and interpreting the live status grid).

## 2.7 Assumptions and Dependencies

- Students have local administrative rights sufficient to install and run Docker containers.
- A reliable internet connection is available to both teachers and students for upload, publishing, and status reporting.
- File storage (S3 or local disk) is available and reliable for storing uploaded PDFs and generated YAML files.
- The chosen LLM parsing service remains available and within acceptable latency/cost bounds throughout the term.
- Pre-built base Docker images used by the Template Library are compatible with the packages typically required by course exams.

## 3. External Interface Requirements

---

### 3.1 User Interfaces

Teacher Dashboard (web): a file-upload screen for exam requirement PDFs, a YAML review/edit screen with a 'Publish' action, and a real-time status grid showing each student as Ready (green), Failed (red), or Not Started (grey), with click-through access to detailed error logs.

Student Agent (CLI): commands to list available environments, pull the published YAML for an exam, display a human-readable dry-run preview of planned changes, and require explicit 'Yes' confirmation before provisioning.

### 3.2 Hardware Interfaces

The Student Agent interfaces with the local machine's Docker Engine to build images and run containers with specified port mappings and volume mounts. No other direct hardware interfaces are required.

### 3.3 Software Interfaces

- **Backend ↔ Database:** PostgreSQL stores users, YAML specs, student status, and logs.
- **Backend ↔ Task Queue:** Celery with a Redis broker handles asynchronous PDF parsing jobs so the teacher does not have to wait for parsing to complete.
- **Backend ↔ ML Parser:** The backend invokes the LLM-based parser service, which returns structured data (packages, versions, environment variables, ports, datasets, setup commands) that the backend converts into YAML.
- **Backend ↔ File Storage:** Uploaded PDFs are stored on S3 or local disk, referenced by the backend for parsing and audit purposes.
- **Student Agent ↔ Docker Engine API:** The agent builds images and runs containers programmatically via the Docker Engine API.

### 3.4 Communications Interfaces

- **Agent–Backend API:** The Student Agent communicates with the Backend Hub over HTTPS to fetch published YAML specs and to POST verification results and logs.
- **Dashboard WebSockets:** The Teacher Dashboard maintains a WebSocket connection to the backend to receive real-time status updates (Ready / Failed / Not Started) without manual refresh.

## 4. System Features

---

### 4.1 Teacher Upload & AI Parsing

#### *Description and Priority*

Allows a teacher to upload a PDF of exam requirements, have it automatically converted into a structured YAML IaC spec by the ML parser, and review/edit the result before publishing. Priority: High.

#### *Stimulus/Response Sequences*

- Teacher uploads a PDF via the dashboard → Backend stores the file and asynchronously triggers the ML Parser via Celery/Redis.
- ML Parser extracts packages, versions, environment variables, ports, datasets, and setup commands, and returns a YAML spec → Backend stores and displays it for review.
- Teacher edits the YAML (if the AI made mistakes) and clicks 'Publish' → Spec becomes available for students to pull.

#### *Functional Requirements*

**REQ-UPLOAD-1:** The system shall allow a teacher to upload an exam requirement document in PDF format via the dashboard.

**REQ-UPLOAD-2:** The system shall process uploaded PDFs asynchronously so the teacher is not blocked while parsing occurs.

**REQ-UPLOAD-3:** The system shall extract packages, versions, environment variables, ports, datasets, and setup commands from the PDF and generate a corresponding YAML specification.

**REQ-UPLOAD-4:** The system shall allow the teacher to view and manually edit the generated YAML before publishing.

**REQ-UPLOAD-5:** The system shall only make a YAML specification available to students after the teacher explicitly publishes it.

### 4.2 Student Environment Provisioning

#### *Description and Priority*

Allows a student to fetch a published environment spec, preview the changes, and provision an isolated, verified Docker environment with one click. Priority: High.

#### *Stimulus/Response Sequences*

- Student runs the Agent CLI → Agent fetches the list of available environments and pulls the published YAML for their exam.
- Agent performs a dry-run and displays a preview of planned changes → Student confirms with 'Yes' to proceed.
- Agent builds a Docker image from the YAML spec and runs the container with specified ports/volumes → A verification script checks all dependencies inside the container.
- Agent reports the verification result and logs back to the Backend via API → Teacher's dashboard updates in real time.

#### *Functional Requirements*

**REQ-AGENT-1:** The system shall provide a CLI agent that authenticates the student and lists available published environments.

**REQ-AGENT-2:** The system shall display a dry-run preview of all changes to be made (packages to install, ports to expose) before provisioning.

**REQ-AGENT-3:** The system shall require explicit student confirmation before building or running any container.

**REQ-AGENT-4:** The system shall build a Docker image from the published YAML spec and run it with the specified exposed ports and volume mounts.

**REQ-AGENT-5:** The system shall run an automated verification script inside the container to confirm that all required dependencies are correctly installed.

**REQ-AGENT-6:** The system shall report the verification outcome (pass/fail) and associated logs back to the backend immediately after provisioning completes.

### 4.3 Real-Time Teacher Monitoring Dashboard

#### *Description and Priority*

Gives teachers a live, per-student view of environment readiness, with the ability to drill into failures and help students debug. Priority: Medium-High.

#### *Stimulus/Response Sequences*

- Student agent reports status to the backend → Backend pushes the update to the teacher's dashboard via WebSocket.
- Teacher clicks a student row marked 'Failed' → Dashboard displays the exact error log (for example, 'Port 5432 already in use') for debugging.

#### *Functional Requirements*

**REQ-DASH-1:** The system shall display, for every student, a real-time status of Ready (green), Failed (red), or Not Started (grey).

**REQ-DASH-2:** The system shall update dashboard status automatically via WebSockets without requiring a manual page refresh.

**REQ-DASH-3:** The system shall allow a teacher to click on a student's row to view the detailed error logs behind a 'Failed' status.

## 5. Other Nonfunctional Requirements

---

### 5.1 Performance Requirements

- PDF parsing shall run asynchronously and shall not block the teacher's dashboard session.
- Dashboard status updates shall be reflected to the teacher within a few seconds of the student agent reporting them, via WebSockets.
- The provisioning flow (dry-run through verification) should complete within a time frame reasonable for an exam setting, dependent on image size and package count.

### 5.2 Safety Requirements

- Student-provisioned environments shall run inside isolated Docker containers to prevent interference with the host system.
- The dry-run preview shall clearly communicate all changes before they are applied, so students are never surprised by an unreviewed action.

### 5.3 Security Requirements

- Teacher and student accounts shall require authentication before accessing upload, review, or provisioning functionality.
- Uploaded files and generated YAML specs shall be stored securely and only be accessible to authorized users (the owning teacher and enrolled students).
- Communication between the Student Agent and the Backend shall occur over HTTPS.
- Containers shall run with least-privilege settings to limit the impact of any misconfigured or malicious spec.

### 5.4 Software Quality Attributes

- **Reliability:** Verification scripts must accurately detect missing or misconfigured dependencies so 'Ready' status can be trusted.
- **Usability:** The dry-run preview and plain-language error logs (e.g., 'Port 5432 already in use') should be understandable by students without deep DevOps background.
- **Portability:** The Student Agent CLI should run consistently across the common desktop operating systems used by students.
- **Maintainability:** The Template Library of pre-built YAML specs and verification scripts should make it easy to support new course types without rebuilding the parser logic.

### 5.5 Business Rules

- Only the owning teacher may edit and publish a YAML specification for their exam.
- Students may only pull and provision environments for exams they are enrolled in.
- A specification must be explicitly published before any student can access it — AI-generated drafts are never visible to students.

## 6. Other Requirements

---

Database backup and retention requirements for uploaded PDFs, generated YAML specs, and verification logs are to be defined during the design phase. Internationalization is not required for the initial release.

## Appendix A: Glossary

---

- **YAML / IaC Spec:** A structured, human-readable file describing the packages, environment variables, ports, datasets, and setup commands required for an exam environment.

- **Dry-Run:** A preview step in which the Student Agent shows planned changes without applying them, pending student confirmation.
- **Student Agent:** The Python CLI tool students run locally to fetch, preview, and provision their exam environment.
- **Celery / Redis:** The asynchronous task queue and message broker used to run PDF parsing jobs in the background.
- **WebSocket:** A persistent two-way connection used to push real-time status updates to the teacher's dashboard.

## Appendix B: Analysis Models

---

See Figure 1 in Section 2.1 for the end-to-end system architecture and data-flow diagram across the four zones (Teacher, Backend Hub, ML Parser, and Student).

## Appendix C: To Be Determined List

---

- TBD-1: Final choice of LLM provider/model for PDF parsing and associated cost/latency limits.
- TBD-2: Maximum supported PDF size and package/dependency count per environment spec.
- TBD-3: Exact SLA for dashboard update latency and provisioning completion time.
- TBD-4: Data retention policy for uploaded PDFs, YAML specs, and verification logs.

## Project Team & Contribution Matrix

---

| Role     | Designation              | Responsibilities                                                                                                                                        |
|----------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Member 1 | ML / Parser Engineer     | Builds the LLM parser; writes prompts to extract packages/ports from PDFs; creates the Template Library (pre-built YAMLS) and verification scripts.     |
| Member 2 | Backend Developer        | Builds the FastAPI server; creates APIs for login, file upload, YAML review, and receiving agent reports; sets up PostgreSQL and Celery/Redis.          |
| Member 3 | DevOps / Agent Developer | Builds the Student Agent (Python CLI); integrates with Docker; writes Dockerfiles for environments; sets up CI/CD and deploys the system on a cloud VM. |